🚀 Project Apex-C: Technical Deployment & Usage Guide:

🚀 **Project Apex-C: Technical Deployment & Usage Guide**

This document provides the necessary instructions to deploy and execute the **Project Apex-C (Server-Side API Exploit Suite)**. Please follow these steps carefully to ensure the automation engine runs correctly.

1. Pre-requisites (Environment Setup)

Before running the script, ensure your system (Kali Linux, Ubuntu, or Windows) has the following components installed:

- **Python 3.x:** To execute the core logic.
- **Google Chrome:** Required for browser automation.
- **Required Libraries:** Open your terminal/command prompt and run:

Bash

pip install undetected-chromedriver selenium

2. File Organization

Your project package should contain the following files:

- final_exploit15.py — The core automation engine.
- Project_Apex_C_Guide.pdf — This instruction manual.
- critical_discovery.log — Reference data for API responses.
- impact_evidence.log — Evidence of successful server-side manipulation.

3. Step-by-Step Execution Process

Step 1: Environment Cleanup

The script is designed to run in a clean environment. Close all existing Chrome instances before starting. The script will attempt to do this automatically by running pkill.

Step 2: Launch the Engine

Run the script using Python:

Bash

python3 final_exploit15.py

A new Chrome window will open in a specialized "Isolation Mode" to bypass standard automation detection.

Step 3: Manual Authentication (Crucial)

Once the browser opens, navigate to the target login page (e.g., web.crypto.com).

- **Note:** The script includes a **120-second sleep timer**. You must complete the login process manually within this window so the engine can capture valid session cookies and Alfred service tokens.

Step 4: Automated Fuzzing & Extraction

After the login timer expires, the engine will automatically:

1. Crawl private API proxy endpoints.
2. Extract sensitive session tokens (as seen in token_show_.jpg).
3. Inject payloads to test for **Privilege Escalation** and **KYC/Limit Bypasses**.

Step 5: Verifying Results

- Check the console output. If a bypass is successful, you will see: [!!!] SERVER LEVEL VULNERABILITY CONFIRMED on [endpoint_path]
- All real-time responses are logged into the .log files for further analysis.

4. Important Safety & Usage Notes

- **Detection Evasion:** The script uses randomized sleep intervals. Do not reduce these values significantly, or you may trigger the platform's Web Application Firewall (WAF).
- **Legal Disclaimer:** This tool is provided for **educational and authorized security research purposes only**. The developer is not responsible for any misuse.